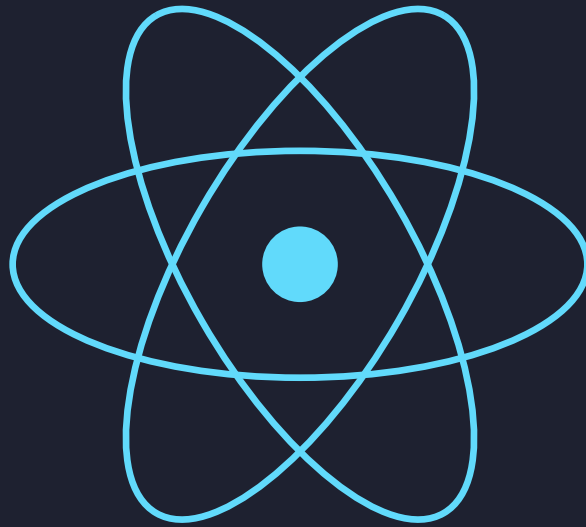




React Hooks

Complete Reference Guide

`useState` • `useEffect` • `useMemo` • `useCallback`

`useContext` • `useRef` • `useReducer`

7 Core Hooks • Deep Dives • Real-world Examples • Use Case Patterns

React 18+ Compatible

-
- 01 — Local Component State
 - 02 — Side Effects & Lifecycle
 - 03 — Memoized Values
 - 04 — Memoized Functions
 - 05 — Context Consumption
 - 06 — Mutable Refs & DOM
 - 07 — Complex State Logic
 - 08 — All Hooks at a Glance

Signature: `const [state, setState] = useState(initialValue)`

useState is the most fundamental React hook. It lets functional components hold and update their own state. Every time you call the setter function, React schedules a re-render with the new value. The initial value is only used on the very first render — subsequent renders ignore it.

When to Use

- Toggle UI elements (modals, dropdowns, sidebars)
- Store form field values and validation errors
- Track loading / error / success states for API calls
- Counter or score values in games / dashboards
- Any isolated piece of data owned by a single component

Gotchas

- ! State updates are asynchronous — don't read the new value immediately after `setState`.
- ! For objects/arrays, always spread or clone: `setState(prev => ({...prev, key: val}))`.
- ! Passing a function to `useState` (lazy initialization) avoids recomputing expensive defaults.
- ! Never mutate state directly — always create new references.

Code Example

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);
  const [name, setName] = useState('');
  const [items, setItems] = useState([]);

  // Functional update – safe when new state depends on old
  const increment = () => setCount(prev => prev + 1);

  // Adding to array without mutation
  const addItem = (item) => setItems(prev => [...prev, item]);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={increment}>+1</button>
      <input value={name} onChange={e => setName(e.target.value)} />
    </div>
  );
}
```

Signature: `useEffect(() => { /* effect */ return cleanup; }, [deps])`



`useEffect` lets you perform side effects after the browser has painted — such as fetching data, subscribing to events, updating the DOM, or setting up timers. The dependency array controls when the effect re-runs. Returning a cleanup function prevents memory leaks by tearing down the effect when the component unmounts or before the next effect runs.

When to Use

- Fetch data from an API when a component mounts or a dependency changes
- Subscribe to WebSocket / EventEmitter and unsubscribe on unmount
- Sync component state to localStorage or a third-party library
- Set up and clear setInterval / setTimeout
- Imperatively update document.title or other DOM globals

Gotchas

- ! `[]` runs once on mount; no array runs after every render; `[x]` runs when `x` changes.
- ! Always add every reactive value used inside the effect to the deps array.
- ! Cleanup function must cancel subscriptions, clear timers, abort fetch to avoid stale effects.
- ! Don't set state unconditionally in an effect without deps — causes infinite loops.
- ! Prefer `useLayoutEffect` when reading / writing layout before the browser paints.

Code Example

```
import { useState, useEffect } from 'react';

function UserProfile({ userId }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    let cancelled = false; // prevent stale state on fast switching
    setLoading(true);

    fetch(`/api/users/${userId}`)
      .then(r => r.json())
      .then(data => {
        if (!cancelled) { setUser(data); setLoading(false); }
      });

    return () => { cancelled = true; }; // cleanup
  }, [userId]); // re-runs whenever userId changes

  if (loading) return <p>Loading...</p>;
  return <h1>{user.name}</h1>;
}
```

Signature: `const result = useMemo(() => expensiveCalc(a, b), [a, b])`

useMemo caches the return value of a function between renders. React only recomputes it when one of the listed dependencies changes. This is purely a performance optimization — your code must be correct without it. Overusing useMemo adds overhead; benchmark before adding it.

When to Use

- Heavy data transformations: sorting/filtering large arrays on every render
- Expensive mathematical computations (statistics, graph traversals)
- Deriving complex objects that are passed to memoized child components
- Avoiding recalculation of selector results in large lists
- Stabilizing reference identity of objects/arrays for downstream deps

Gotchas

- ! Don't use useMemo for simple calculations — the memoization overhead costs more.
- ! If the factory function has side effects, use useEffect instead.
- ! Dependencies must be exhaustive — missing a dep causes stale cached values.
- ! React may discard cached values (e.g., in concurrent mode) — don't rely on it for correctness.

Code Example

```
import { useState, useMemo } from 'react';

function ProductList({ products, query, sortBy }) {
  // Only re-runs when products, query, or sortBy changes
  const filtered = useMemo(() => {
    console.log('filtering...'); // won't spam every keystroke
    return products
      .filter(p => p.name.toLowerCase().includes(query.toLowerCase()))
      .sort((a, b) =>
        sortBy === 'price' ? a.price - b.price
        : a.name.localeCompare(b.name)
      );
  }, [products, query, sortBy]);

  return (
    <ul>
      {filtered.map(p => <li key={p.id}>{p.name} - ${p.price}</li>)}
    </ul>
  );
}
```

Signature: `const fn = useCallback(() => doSomething(a, b), [a, b])`

useCallback memoizes a function itself (not its return value). It returns the same function reference between renders unless a dependency changes. This is mainly useful when passing callbacks to heavily optimized child components wrapped in `React.memo`, or when a function is listed in `useEffect` dependencies and would otherwise trigger the effect on every render.

When to Use

- Passing event handlers to `React.memo` child components to prevent re-renders
- Stabilizing function references used in `useEffect` / `useMemo` deps arrays
- Callbacks passed to custom hooks that internally compare references
- Handlers in virtualized lists (`react-window` / `react-virtual`)
- Debounced / throttled functions that need a stable identity

Gotchas

- ! `useCallback` alone does nothing — it only helps if the child uses `React.memo`.
- ! All values from the component scope used inside must be in deps.
- ! `useCallback(fn, [])` with an empty dep array captures the initial closure — watch for stale state.
- ! `useCallback` is the function twin of `useMemo`:
`useCallback(fn, deps) === useMemo(() => fn, deps)`.

Code Example

```
import { useState, useCallback, memo } from 'react';

// Memoized child – only re-renders when its own props change
const Button = memo(({ onClick, label }) => {
  console.log('Button rendered:', label);
  return <button onClick={onClick}>{label}</button>;
});

function Parent() {
  const [count, setCount] = useState(0);
  const [other, setOther] = useState(0);

  // Stable reference – won't cause Button to re-render
  const handleIncrement = useCallback(() => {
    setCount(prev => prev + 1);
  }, []); // no deps – uses functional update

  return (
    <div>
      <p>Count: {count} Other: {other}</p>
      <Button onClick={handleIncrement} label='Increment' />
      <button onClick={() => setOther(o => o + 1)}>Other</button>
    </div>
  );
}
```


Signature: `const value = useContext(MyContext)`



`useContext` reads the current value of a Context and subscribes the component to future updates. When the nearest Provider above in the tree updates its value, all consumers re-render. Context is ideal for truly global data like authentication, themes, language (i18n), or feature flags.

When to Use

- Authentication state (current user, roles, tokens) across the whole app
- Theme / dark-mode toggling across deeply nested components
- Language / locale settings for internationalization
- Shopping cart state shared between header, product page, and checkout
- Feature flags or app-wide configuration values

Gotchas

- ! Every consumer re-renders when the context value changes — memoize wisely.
- ! Split large contexts into smaller ones to limit unnecessary re-renders.
- ! Context is not a replacement for all state — it suits 'ambient' data.
- ! Default value is only used when there is no Provider above in the tree.

Code Example

```
import { createContext, useContext, useState } from 'react';

// 1. Create context with a default value
const ThemeContext = createContext({ theme: 'light', toggle: () => {} });

// 2. Provider wraps the tree
export function ThemeProvider({ children }) {
  const [theme, setTheme] = useState('light');
  const toggle = () => setTheme(t => t === 'light' ? 'dark' : 'light');
  return (
    <ThemeContext.Provider value={{ theme, toggle }}>
      {children}
    </ThemeContext.Provider>
  );
}

// 3. Any descendant consumes without prop drilling
function Navbar() {
  const { theme, toggle } = useContext(ThemeContext);
  return (
    <nav className={theme}>
      <button onClick={toggle}>Switch to {theme === 'light' ? 'dark' : 'light'}</button>
    </nav>
  );
}
```

Signature: `const ref = useRef(initialValue)`



`useRef` returns a mutable object `{ current: initialValue }` that persists for the full lifetime of the component. Unlike state, mutating `ref.current` does not trigger a re-render. It has two primary uses: accessing DOM elements directly (focus, measurements, media controls) and storing any mutable value that must survive re-renders without causing them.

When to Use

- Focusing an input, scrolling a container, playing/pausing a video element
- Storing the previous value of a prop or state for comparison
- Keeping a `setInterval` / `setTimeout` ID for later cancellation
- Tracking whether a component is still mounted in async callbacks
- Integrating with third-party imperative libraries (charts, maps, editors)

Gotchas

- ! Changing `ref.current` never triggers a re-render — don't use it as state.
- ! Don't read or write refs during rendering — always in effects or handlers.
- ! Callback refs `((node) => ...)` are more reliable for dynamic elements than ref objects.
- ! Forwarding refs to child components requires `React.forwardRef`.

Code Example

```
import { useRef, useEffect, useState } from 'react';

function Stopwatch() {
  const [time, setTime] = useState(0);
  const [running, setRunning] = useState(false);
  const intervalRef = useRef(null); // stores timer id
  const inputRef = useRef(null); // DOM reference

  useEffect(() => {
    if (running) {
      intervalRef.current = setInterval(() => setTime(t => t + 1), 1000);
    } else {
      clearInterval(intervalRef.current);
    }
    return () => clearInterval(intervalRef.current);
  }, [running]);

  const focusInput = () => inputRef.current?.focus();

  return (
    <div>
      <p>{time}s</p>
      <button onClick={() => setRunning(r => !r)}>
        {running ? 'Pause' : 'Start'}
      </button>
      <input ref={inputRef} placeholder='Type here' />
      <button onClick={focusInput}>Focus Input</button>
    </div>
  );
}
```

Signature: `const [state, dispatch] = useReducer(reducer, initialState)`

useReducer is an alternative to useState for complex state logic. Instead of multiple setState calls, you dispatch action objects that are handled by a pure reducer function. This makes state transitions explicit, testable, and easy to reason about — especially when multiple pieces of state depend on each other. It follows the same pattern popularized by Redux.

When to Use

- Form with many fields where changes to one field affect others
- Complex UI state machines (wizard steps, multi-stage flows)
- Shopping cart: add, remove, update quantity, apply coupon, clear
- Async fetch state with loading / success / error phases
- When next state depends on multiple parts of the current state

Gotchas

- ! Reducer must be a pure function — no side effects, no async, no mutations.
- ! Dispatch is stable across renders — safe to omit from useEffect deps.
- ! Prefer useReducer over useState when you have 3+ related boolean flags.
- ! Can combine with useContext to build a lightweight Redux-style global store.

Code Example

```
import { useReducer } from 'react';

const initialState = { items: [], total: 0, coupon: null };

function cartReducer(state, action) {
  switch (action.type) {
    case 'ADD_ITEM': {
      const exists = state.items.find(i => i.id === action.item.id);
      const items = exists
        ? state.items.map(i => i.id === action.item.id
          ? { ...i, qty: i.qty + 1 } : i)
        : [...state.items, { ...action.item, qty: 1 }];
      return { ...state, items, total: items.reduce((s,i) => s + i.price*i.qty, 0) };
    }
    case 'REMOVE_ITEM': {
      const items = state.items.filter(i => i.id !== action.id);
      return { ...state, items, total: items.reduce((s,i) => s + i.price*i.qty, 0) };
    }
    case 'APPLY_COUPON':
      return { ...state, coupon: action.code };
    case 'CLEAR_CART':
      return initialState;
    default:
      return state;
  }
}

function Cart() {
  const [cart, dispatch] = useReducer(cartReducer, initialState);
  const addProduct = (p) => dispatch({ type: 'ADD_ITEM', item: p });
  const clearAll = () => dispatch({ type: 'CLEAR_CART' });
  return <p>Items: {cart.items.length} | Total: ${cart.total}</p>;
}
```

Hook	Returns	Triggers Re-render?	Primary Purpose
<code>useState</code>	value, setter	Yes	Simple local state
<code>useEffect</code>	void	No	Side effects / sync
<code>useMemo</code>	cached value	No	Memoize computations
<code>useCallback</code>	cached fn	No	Memoize functions
<code>useContext</code>	context value	Yes (if ctx changes)	Consume global state
<code>useRef</code>	{ current }	No	Mutable ref / DOM access
<code>useReducer</code>	state, dispatch	Yes	Complex state machine

■ **Rule of thumb:** Start with `useState`. Reach for `useReducer` when state grows complex. Add `useMemo` / `useCallback` only after profiling reveals a bottleneck. Use `useContext` for global ambient data, not server cache.